Stajyer Yerleştirme Simülatörü: Greedy ve Heuristic Yaklaşımların Karşılaştırmalı Analizi

Sudenur Elmas, Hasna Şahinoğlu

Bilgisayar Mühendisliği, Bursa Teknik Üniversitesi,
Bursa, Türkiye

22360859002@ogr.btu.edu.tr

22360859005@ogr.btu.edu.tr

***Öz*— Bu çalışmada, stajyer yerleştirme problemine iki farklı algoritmik yaklaşım uygulanmış ve karşılaştırmalı analizi yapılmıştır. Problem, kısıtlı kontenjanlar altında öğrenci-firma eşleştirmesini, firma red süreçlerini ve iteratif yeniden atama mekanizmalarını içermektedir. Açgözlü (Greedy) algoritma ve Beam Search tabanlı sezgisel (heuristic) yöntem kullanılarak simülasyonlar gerçekleştirilmiştir. Greedy algoritma, öğrencileri GNO sırasına göre işleyerek deterministik atamalar yaparken, Beam Search yöntemi çözüm uzayını daha geniş araştırarak arkadaşlık faktörünü optimize etmektedir. Deneysel sonuçlar, Beam Search yönteminin Greedy algoritmasına göre daha yüksek toplam memnuniyet skoru sağladığını ancak hesaplama süresinin arttığını göstermiştir. Çalışmada ayrıca, Beam Width parametresinin performansa etkisi analiz edilmiş ve GUI ile interaktif görselleştirme sistemi geliştirilmiştir.**

***Anahtar Kelimeler*— Stajyer yerleştirme, açgözlü algoritma, Beam Search, sezgisel optimizasyon**

I. Giriş

Stajyer yerleştirme problemi, firmalar ile farklı tercihlere sahip öğrenciler arasında optimal eşleştirme yapmayı hedefleyen bir problemdir. Günümüzde üniversitelerin staj koordinasyon merkezleri, mezuniyet öncesi zorunlu staj süreçlerini yürütürken benzer yapıdaki problemlerle karşılaşmaktadır. Bu çalışmada ele alınan problem, gerçek dünya yerleştirme sistemlerinin temel özelliklerini içermektedir: (i) her firmanın sabit kontenjan kısıtı, (ii) öğrencilerin tercih sıralamaları, (iii) firmaların red mekanizmaları, (iv) reddedilen öğrenciler için iteratif yeniden atama döngüleri.

Öğrenciler başlangıçta 5 firmayı tercih listesine alırken, reddedildikçe tercihlerini güncellemek zorundadırlar. Firmalar ise kendilerine atanan öğrencileri GNO ve mülakat performansına dayalı bir eşik değeri üzerinden değerlendirerek kabul/red kararı vermektedir. Bu dinamik süreç, her iterasyonda kontenjan ve tercih durumlarının değişmesine neden olmaktadır.

Problem ayrıca, öğrencilerin sosyal faktörlerini de dikkate almaktadır. Her öğrencinin arkadaş listesi ve arkadaş toleransı parametreleri bulunmakta, aynı firmaya arkadaşlarıyla yerleşmeleri durumunda memnuniyet skorları artmaktadır. Bu özellik, problemi tek boyutlu bir tercih-kontenjan eşleştirmesinin ötesine taşıyarak çok kriterli bir optimizasyon haline getirmektedir.

II. Problem Modelleme

Stajyer yerleştirme problemi; öğrenciler, firmalar, tercihler, kontenjanlar ve memnuniyet ölçütleri üzerinden biçimsel olarak tanımlanır; çalışmada kullanılan değerlendirme ve red mekanizmaları aşağıda açıklanmaktadır.

A. Problem Tanımı

Problem, sonlu sayıda öğrenci ve firmadan oluşacak şekilde ele alınmıştır. Her firmanın sınırlı bir kontenjanı bulunmaktadır. Her öğrenci, firmalar arasından sıralı ve sınırlı bir tercih listesine sahiptir.

Yerleştirme süreci iteratif bir yapıdadır. Her iterasyonda, henüz kalıcı olarak yerleşmemiş öğrenciler değerlendirilir ve tercih listelerinde yer alan firmalara atanır. Firmalar, kendilerine atanan öğrencilere bir değerlendirme puanı verir ve bu skora göre öğrenciyi kabul eder veya reddeder. Reddedilen öğrenciler bir sonraki iterasyonda tercih listelerini güncelleyerek sürece yeniden dahil olur.

Amaç, tüm öğrencilerin yerleştirilmesini sağlarken sistem genelinde tanımlanan toplam memnuniyet skorunu maksimize etmektir.

B. Memnuniyet Skoru Fonksiyonu

Çalışmada hem Greedy hem de Heuristic algoritmaları aynı memnuniyet fonksiyonu ile değerlendirilmiştir. Böylece algoritmalar arası karşılaştırma adil bir zemin üzerinde gerçekleştirilmiştir. Memnuniyet skoru, bireysel öğrenci memnuniyetlerinin toplamı olarak tanımlanmıştır.

1) Standart Puanlama:

Bir öğrencinin memnuniyeti yalnızca kalıcı olarak yerleşmiş olması durumunda hesaplanmaktadır. Yerleşememiş veya pas geçilmiş öğrencilerin memnuniyet skoru sıfır kabul edilmiştir.

Her iki algoritma için ortak bir memnuniyet skoru fonksiyonu tanımlanmıştır. Bir öğrencinin memnuniyeti, tercih puanı ve arkadaş bonusu olmak üzere iki bileşenden oluşur:

$$S(i) = P_{\text{tercih}}(s,t) + P_{\text{arkadaş}}(i,k)$$

Burada S(i) i. öğrencinin toplam skoru, s tercih sırası, t yerleşildiği iterasyon, k aynı firmadaki arkadaş sayısıdır.

2) Tercih Puanı Formülü

Öğrencinin yerleştiği firmanın tercih listesindeki sırası temel alınarak bir tercih puanı hesaplanmaktadır. Daha üst sıralardaki tercihler daha yüksek puan üretmektedir.

Yerleştirmenin gerçekleştiği iterasyon sayısı arttıkça, tercih puanı zamansal bir azalma (decay) katsayısı ile çarpılmaktadır. Bu mekanizma, erken iterasyonlarda yapılan yerleştirmelerin daha değerli kabul edilmesini sağlar ve gecikmeli yerleşmeleri doğal olarak cezalandırır.

$$P_{tercih}(s,t) = W_{tercih} \times (MAX_{tercih} + 1 - s) \times (1-\alpha)^{(t}$$

Sistem parametreleri W_tercih = 10, MAX_tercih = 5 ve α = 0.20 olarak belirlenmiştir. Decay faktörü (1 - α)^(t-1), her yeni iterasyonda puanı %20 oranında azaltarak erken yerleşmeyi teşvik eder. Örneğin, 1. tercihine 1. turda yerleşen bir öğrenci 50 puan alırken, aynı tercih 2. turda 40 puan değerindedir.

3) Arkadaş Bonus Sistemi

Modelde, aynı firmaya yerleşen arkadaşların öğrenci memnuniyetini artırdığı varsayımı benimsenmiştir. Bu doğrultuda, bir öğrencinin yerleştiği firmada bulunan arkadaş sayısına bağlı olarak ek bir bonus puanı hesaplanmaktadır. Arkadaş katkısı belirli bir üst sınırla kısıtlanmıştır; böylece memnuniyet skorunun yalnızca arkadaş faktörü üzerinden kontrolsüz biçimde artması engellenmiştir.

Arkadaş bonusu da tercih puanında olduğu gibi iterasyon sayısına bağlı bir azalma katsayısı ile ölçeklendirilmiştir. Bu sayede, geç iterasyonlarda oluşan arkadaş eşleşmelerinin etkisi dengelenmiştir.

Arkadaş puanı, sosyal faktörleri optimizasyon sürecine dahil eder. Formül, tercih sırasına göre iki farklı davranış sergiler:

$$P_{\text{arkadaş}} = \begin{cases} k \times 10, & \text{eğer } s = 1 \\ (s-1) \times 10 \times (0.8)^{(t-1)} \times \min(k,1) & \\ \quad +(k-1) \times 10 \times \min(k,1), & \text{eğer } s > 1 \end{cases}$$

Formülde s, öğrencinin yerleştiği tercih sırası; k, öğrenci ile aynı firmaya atanmış arkadaş sayısını ifade eder.

Birinci tercih durumu (s = 1): Öğrenci en çok istediği firmaya yerleşmişse, her arkadaş için sabit 10 puan bonus eklenir. Bu durumda k × 10 puan kazanır.

Diğer tercihler (s > 1): İki bileşenli bir hesaplama yapılır:

- Temel bonus: Eğer en az bir arkadaş varsa [min(k,1) = 1], tercih sırasına göre (s-1) × 10(0.8)^(t-1) puanı kazanır. Bu terim, düşük sıra tercihlerde daha az ödül verir. Bu sayede öğrencinin arkadaş toleransı değeri aralığındaki tercih puanı, burdan kazanacağı arkadaş puanıyla toplandığında ilk tercih puanına denk olur.
- Ek arkadaş bonusu: İlk arkadaştan sonraki her arkadaş için 10 puan eklenir: (k-1) × 10.

min(k,1) fonksiyonu, arkadaş yoksa (k=0) tüm arkadaş bonusunu sıfırlayan bir anahtar görevi görür. Bu sayede, arkadaşı ile aynı yere atanmayan öğrenciler yalnızca tercih puanından faydalanır.

Tasarım Mantığı: Bu asimetrik yapı, şu prensiplere dayanır:

- En üst tercihe (1. tercih) yerleşen öğrenciler zaten mutludur, bu yüzden her arkadaş eşit değerdedir.
- Alt sıra tercihlere yerleşenler için ilk arkadaşta temel bonus verilir.
- Sonraki arkadaşlar, marjinal olarak daha az değerlidir ancak yine de katkı sağlar.

Sayısal Örnekler:

- 1. tercih, 1. tur, 2 arkadaş:
  P_arkadaş = 2 × 10 = 20 puan
- 3. tercih, 1. tur, 1 arkadaş:
  P_arkadaş = (3-1) × 10 × 1 + (1-1) × 10 × 1 = 20 puan
- 2. tercih, 2. tur, 3 arkadaş:
  P_arkadaş = (2-1) × 8 × 1 + (3-1) × 10 × 1 = 28 puan
  (Not: 2. turda m(2) = 10 × 0.8 = 8)
- 4. tercih, 1. tur, 0 arkadaş:
  P_arkadaş = 0 puan (min(0,1) = 0 nedeniyle tüm terimler sıfır)

4) Sistem Toplam Skoru Hesaplanması

Sistemin toplam memnuniyet skoru, kalıcı olarak yerleşmiş tüm öğrencilerin bireysel memnuniyet skorlarının toplamı olarak hesaplanmaktadır. Bu değer, deneysel çalışmalarda algoritmaların performansını karşılaştırmak için temel ölçüt olarak kullanılmıştır.

$$S_{\text{toplam}} = \sum_{i \in O_{\text{yerleşen}}} S(i) \qquad S_{\text{ortalama}} = \frac{S_{\text{toplam}}}{N_{\text{yerleşen}}}$$

O_yerleşen kümesi, simülasyon sonunda kalıcı atama almış öğrencileri içerir. S_toplam metriği, algoritmaların genel optimizasyon başarısını ölçerken, S_ortalama öğrenci başına ortalama memnuniyeti gösterir.

C. Firma Değerlendirme ve Red Mekanizması

Firma kavramı, klasik başvuru-temelli eşleştirme sistemlerinden farklı olarak büyük ölçüde pasif bir yapıdadır. Öğrencileri yerleştirme işlemi sistem tarafından merkezi olarak yürütülmektedir. Firmaların sürece aktif katkısı, kendilerine atanan öğrenciler için bir kabul puanı üzerinden red veya kabul kararı vermekle sınırlıdır.

Yerleştirme sürecinin ilgili aşamasında, sistem tarafından bir firmaya atanmış olan öğrenciler için firma bazlı bir değerlendirme skoru hesaplanır. Bu skor, öğrencinin akademik başarısını (GNO) ve kodda tanımlanan mülakat puanı bileşenini içeren birleşik bir ölçüttür. Hesaplanan skor, ilgili firmanın eşik değeri ile karşılaştırılır.

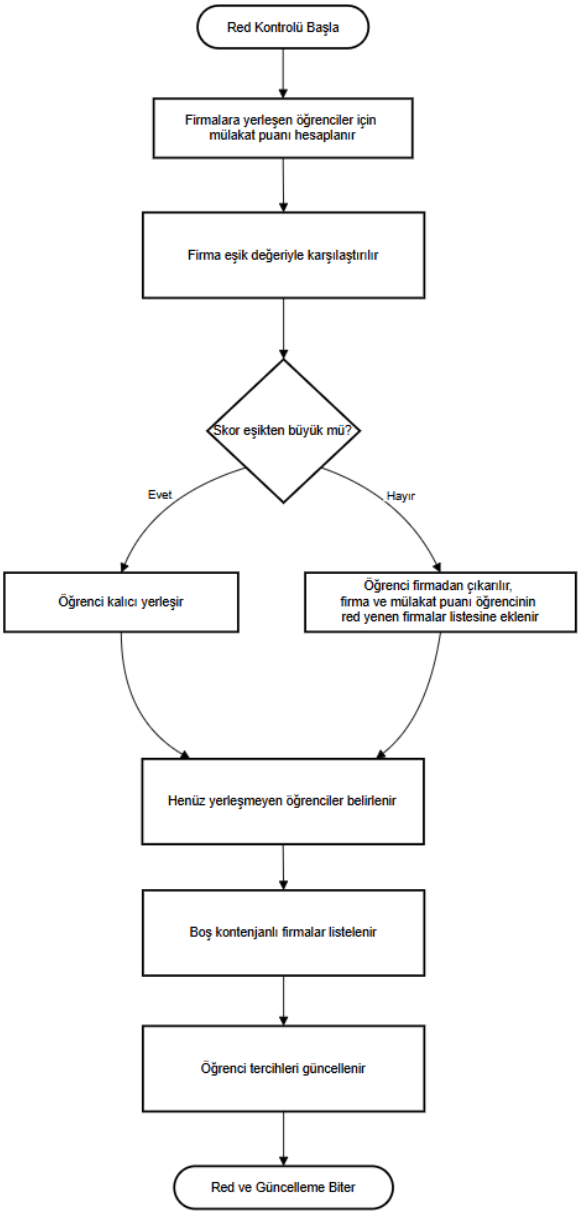
$$Skor_{ij} = 0.5 \times (GNO_i \times 25) + 0.5 \times \text{Random}(0,100)$$

Eşik değerinin üzerinde kalan öğrenciler, firmaya kalıcı olarak yerleşmiş kabul edilir ve sistemdeki durumları sabitlenir. Skoru eşik değerin altında kalan öğrenciler ise firmadan reddedilir. Red durumunda öğrenci, ilgili firmadan çıkarılır ve hem firma bilgisi hem de o firmaya ait mülakat puanı öğrencinin "red yenen firmalar" listesine eklenir. Bu

liste, öğrencinin ilerleyen iterasyonlarda aynı firmayı tekrar tercih etmesini engellemek amacıyla kullanılmaktadır.

Red ve kabul işlemleri tamamlandıktan sonra, henüz kalıcı olarak yerleşememiş öğrenciler yeniden belirlenir. Sistem, boş kontenjana sahip firmaları tespit eder ve bu öğrencilerin tercih listelerini, red yedikleri firmalar tercih listelerine eklenmeyecek şekilde günceller. Güncellenen tercihler ile bir sonraki iterasyonda yerleştirme süreci devam eder.

Bu mekanizma sayesinde firmalar, yalnızca eşik tabanlı bir filtreleme rolü üstlenirken; yerleştirme, red sonrası güncelleme ve iteratif ilerleme tamamen sistem kontrolünde gerçekleştirilmektedir. Red kontrol sürecinin ayrıntılı akışı, ilgili akış şeması Şekil 1 üzerinde adım adım gösterilmektedir.



Şekil 1. Firma Öğrenci Reddetme Mekanizması

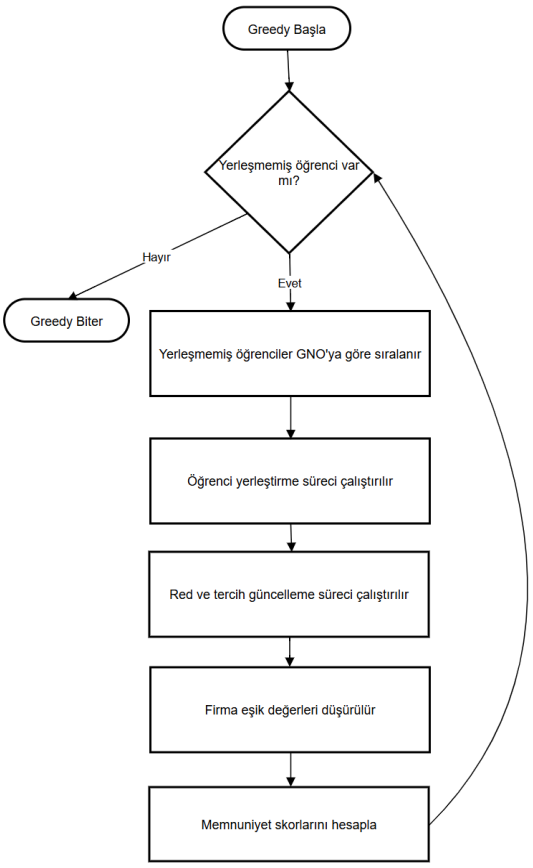
III. Greedy Algoritma

A. Algoritmanın Genel Yapısı

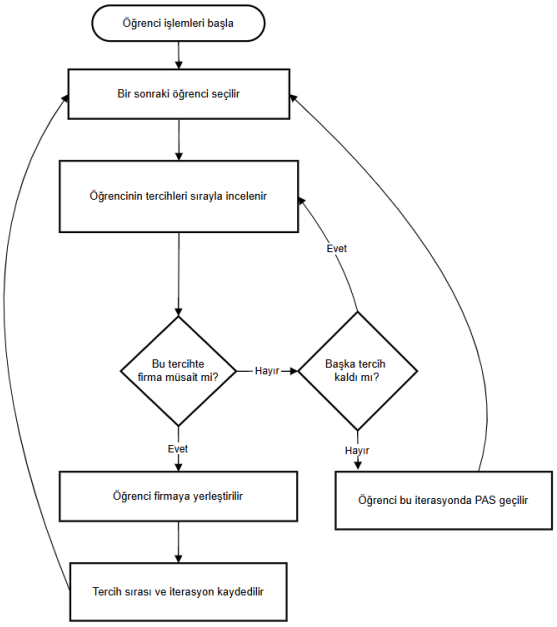
Greedy algoritma, yerleştirme sürecini iteratif olarak yürüten deterministik bir yaklaşımdır. Algoritma, sistemde henüz kalıcı olarak yerleşmemiş öğrenci bulunduğu sürece çalışmaya devam eder. Her iterasyonun başında, yerleşmemiş öğrenciler akademik başarılarına göre azalan biçimde (GNO öncelikli) sıralanır. Bu sıralama, daha yüksek akademik başarıya sahip öğrencilerin erken iterasyonlarda daha avantajlı konumda değerlendirilmesini sağlamaktadır.

Sıralama işleminin ardından, her öğrenci için yerleştirme süreci başlatılır. Öğrenciler, tercih listelerinde yer alan firmalara sırayla atanmaya çalışılır. Uygun kontenjana sahip bir firma bulunduğunda öğrenci geçici olarak bu firmaya yerleştirilir ve ilgili tercih sırası ile iterasyon bilgisi kayıt altına alınır.

Yerleştirme adımı tamamlandıktan sonra, tüm geçici yerleşmeler için firma değerlendirme ve red mekanizması çalıştırılır. Bu süreçte, firmaların eşik değerleri dikkate alınarak öğrenciler kalıcı olarak kabul edilir veya reddedilir. Red sonrası tercih güncelleme ve firma eşik değerlerinin düşürülmesi adımları uygulanarak bir sonraki iterasyona geçilir. İlgili iterasyon sonunda memnuniyet skorları hesaplanır ve sistemde yerleşmemiş öğrenci kalıp kalmadığı kontrol edilir. Greedy algoritmanın genel akışı, Greedy akış şemasında gösterilmektedir.



Şekil 2. Greedy Algoritması Genel Akışı



Şekil 3. Greedy Algoritmasında Öğrenci Yerleştirme Süreci

B. Tercih Güncelleme Stratejisi

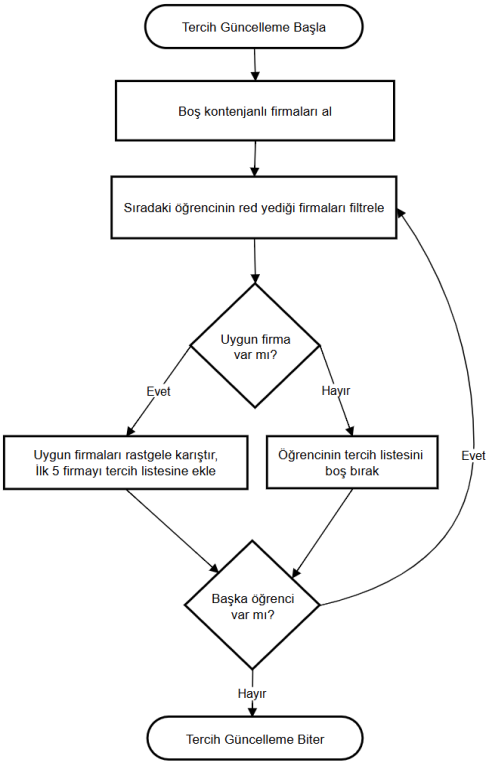
Greedy yaklaşımda tercih güncelleme, öğrencinin sonraki iterasyona kaldığı durumlarda devreye giren dinamik bir mekanizmadır. Bir öğrenci, bir firmadan red aldığında, bu firma öğrencinin red yenen firmalar listesine eklenir. Bir sonraki iterasyonda, öğrencinin tercih listesi yeniden oluşturulurken bu firmalar listeye alınmaz. Böylece öğrencinin aynı firmaya tekrar atanmasının önüne geçilir.

Yerleştirme sürecinde bir öğrencinin tüm tercihleri dolduysa, öğrenci ilgili iterasyonda pas geçilir. Pas geçen öğrenciler, sonraki iterasyonlarda güncellenmiş firma ve kontenjan durumlarına göre yeniden tercih yapar. Bu strateji, algoritmanın çıkmaza girmeden ilerlemesini ve tercih havuzunun zamanla genişlemesini sağlar.

C. Zorunlu Yerleştirme Mekanizması

Bu mekanizma, tercih havuzunun tükenmesi durumunda devreye giren bir mekanizma olarak tasarlanmıştır. Belirli bir iterasyon sonunda bir öğrenci hâlâ yerleşememişse ve sistemde boş kontenjan bulunmasına rağmen bu öğrenci daha önce tüm bu firmalardan red almışsa, öğrenci bir sonraki iterasyon için geçerli bir tercih listesi oluşturamaz. Bu durumda algoritma, yerleştirme sürecinin çıkmaza girmesini önlemek amacıyla zorunlu yerleştirme adımını uygular.

Sistem, öğrencinin red yediği firmalar listesini inceleyerek hâlen kontenjanı bulunan firmaları belirler. Bu firmalar arasından, öğrencinin en yüksek red puanını aldığı firma seçilir ve öğrenci bu firmaya doğrudan atanır. Böylece zorunlu yerleştirme kararı rastgele değil, öğrencinin önceki değerlendirmelerde aldığı en yüksek puana göre verilir. Bu mekanizma sayesinde tüm öğrenciler için yerleşme garantisi sağlanabilmektedir.



Şekil 4. Greedy Algoritmasında Tercih Güncelleme Süreci

D. Zaman Karmaşıklığı Analizi

Greedy algoritmanın zaman karmaşıklığı, iterasyon sayısı, öğrenci sayısı ve tercih listesi uzunluğu ile doğrudan ilişkilidir. Her iterasyonda yerleşmemiş öğrencilerin sıralanması işlemi O(N log N) karmaşıklığındadır. Ardından her öğrenci için tercih listesinin taranması en kötü durumda O(Tmax) zaman almaktadır.

Firma değerlendirme ve red mekanizması ise her iterasyonda yeni yerleşen öğrenciler üzerinden çalışmakta ve toplamda doğrusal bir ek maliyet oluşturmaktadır. İterasyon sayısının sınırlı ve pratikte düşük tutulması nedeniyle, Greedy algoritma deneysel çalışmalarda makul çalışma süreleri sunmuş; buna karşılık deterministik yapısı nedeniyle çözüm kalitesi açısından sezgisel yaklaşımlara kıyasla sınırlı kalmıştır.

E. Greedy Yaklaşımda Puanlama ve Sonuçların Değerlendirilmesi

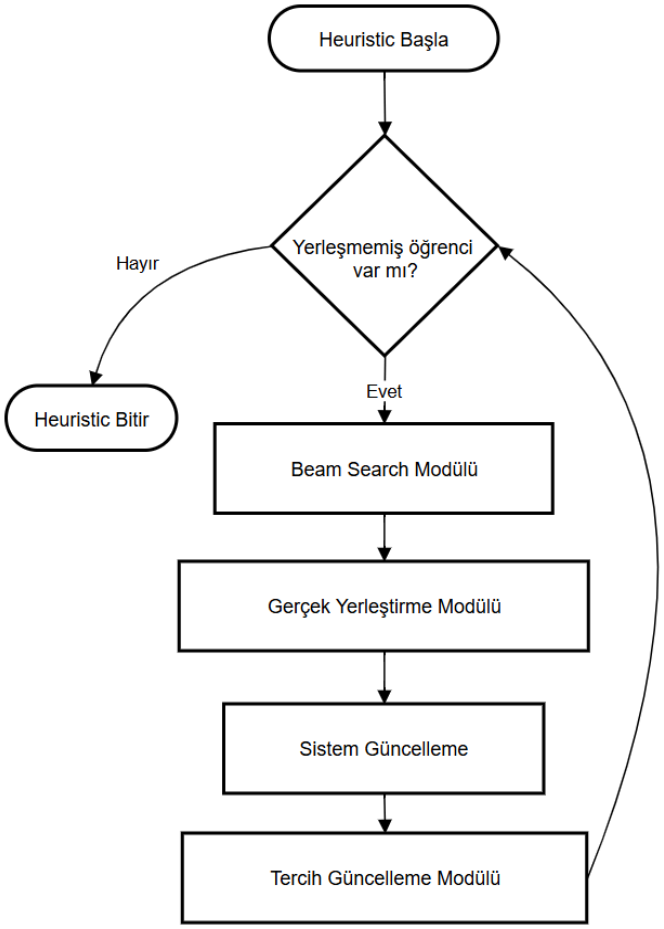
Greedy algoritma da arkadaş faktörü, yerleştirme sürecini yönlendiren aktif bir karar değişkeni olarak kullanılmamaktadır. Algoritma, öğrencilerin firmalara atanması sırasında yalnızca GNO sıralaması, tercih listeleri ve mevcut kontenjan durumunu dikkate almaktadır. Arkadaşlık bilgisi yerleştirme tamamlandıktan sonra memnuniyet skorunun hesaplanması sürecinde değerlendirilmekte ve yalnızca sonuç metriğine katkı sağlamaktadır.

Bu durum, Greedy algoritmanın hesaplama açısından verimli ve basit olmasını sağlarken, arkadaşlık gibi sosyal faktörleri optimize etme konusunda yetersiz kalmasına neden olmaktadır. Bu sınırlılık, çözüm uzayını daha geniş biçimde araştırabilen ve memnuniyet skorunu doğrudan optimize etmeyi hedefleyen sezgisel yaklaşımların kullanılmasını

gerekli kılmaktadır. Bu motivasyon doğrultusunda, bir sonraki bölümde Beam Search tabanlı heuristic yerleştirme yöntemi ayrıntılı olarak ele alınmaktadır.

IV. Heuristic Yaklaşım

Bu bölümde Beam Search tabanlı sezgisel (heuristic) yaklaşım ayrıntılı biçimde açıklanmaktadır. Heuristic yöntem, Greedy algoritmadan farklı olarak, yerleştirme sürecinde birden fazla olası çözümü eş zamanlı olarak değerlendirerek toplam memnuniyet skorunu artırmayı hedeflemektedir.



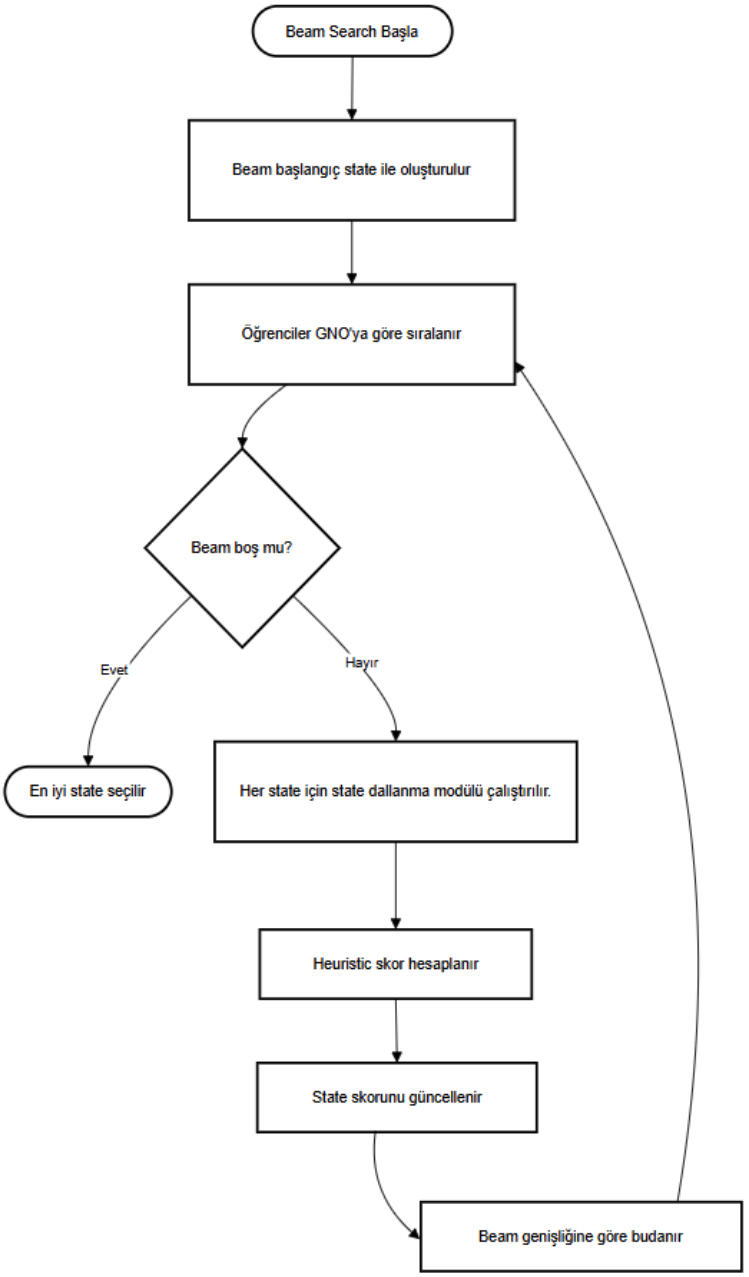
Şekil 5. Heuristic Yaklaşımın Genel Algoritması

A. Beam Search Teorisi

Beam Search, çözüm uzayını kontrollü biçimde tarayan bir sezgisel arama yöntemidir. Bu yaklaşımda sistem, tek bir yerleştirme senaryosu yerine, her iterasyonda birden fazla aday çözümü (state) paralel olarak değerlendirir. Her state, o ana kadar oluşturulmuş kısmi yerleştirmeleri, öğrencilerin durumlarını ve ilgili skor bilgilerini içermektedir.

Algoritma, başlangıçta tek bir başlangıç state'i ile başlatılır. Daha sonra öğrenciler GNO değerlerine göre sıralanır ve Beam Search döngüsü çalıştırılır. Her iterasyonda mevcut beam içindeki state'ler genişletilir, sezgisel bir skorla değerlendirilir ve yalnızca en iyi skor değerine sahip belirli sayıdaki state (beam width) korunur. Beam boşaldığında, en yüksek skora sahip state algoritmanın çıktısı olarak seçilir.

Bu yapı sayesinde, çözüm uzayı tamamen taranmadan, umut vadeden çözümler üzerinde yoğunlaşılmış olur.



Şekil 6. Beam Search Algoritması

B. Beam Width Parametresi

Beam width parametresi, her iterasyonda korunacak maksimum state sayısını belirlemektedir. Küçük beam width değerleri, algoritmayı daha hızlı ancak daha sınırlı bir arama yapısına iterken; büyük beam width değerleri çözüm uzayının daha geniş taranmasını sağlamakta ve genellikle daha yüksek memnuniyet skorları üretmektedir.

C. Genişletme Fonksiyonu

Heuristic yaklaşımda genişletme fonksiyonu, Beam Search algoritmasının çözüm kalitesini belirleyen temel bileşendir. Bu fonksiyon, Greedy algoritmadaki tekil ve deterministik yerleştirme kararlarının aksine, bir öğrencinin birden fazla olası yerleştirme senaryosunu aynı anda değerlendirebilmesini sağlar. Her state genişletme adımı, sistemin o ana kadarki yerleşim durumunu temel alarak yeni alternatif state'ler üretir ve böylece çözüm uzayı kontrollü biçimde genişletilir.

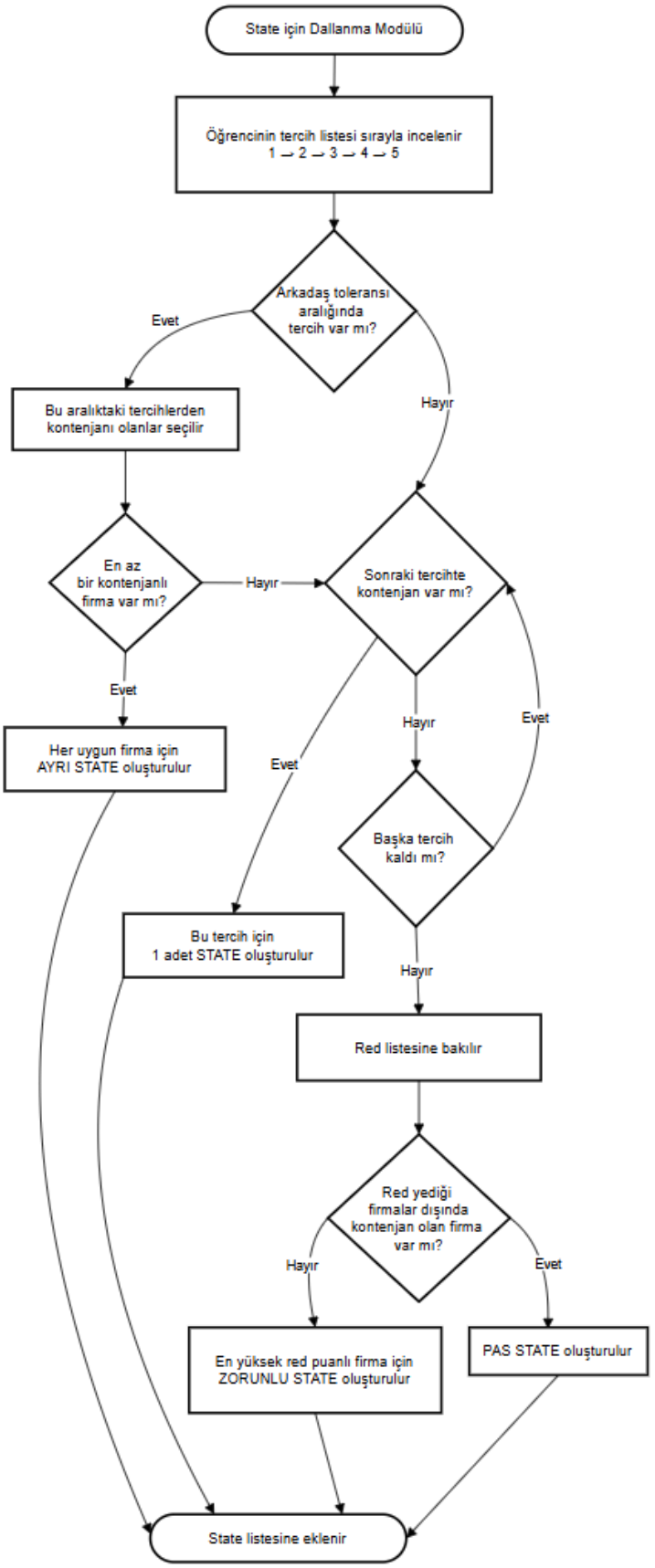
Genişletme sürecinde ilk olarak öğrencinin arkadaşlık toleransı parametresi dikkate alınır. Arkadaşlık toleransı, bir öğrencinin arkadaşlarıyla aynı firmaya yerleşebilmek adına tercih listesindeki kaç basamak aşağı inmeyi kabul edebileceğini ifade eden bir eşiktir. Örneğin tolerans değeri 2 olan bir öğrenci için ilk üç tercih, sosyal açıdan kabul edilebilir alternatifler olarak değerlendirilir. Bu parametre, sosyal faktörlerin yerleştirme kararlarına bilinçli ve sınırlı bir biçimde dahil edilmesini sağlar.

Arkadaşlık toleransı ile memnuniyet puanı mekanizması birlikte ele alındığında, tolerans aralığındaki alt tercihler, üst tercihlerle rekabet edebilir hale gelmektedir. Tercih puanları alt sıralara doğru azalırken, aynı firmada bulunan arkadaşlar için verilen bonus puanlar bu farkı dengelemekte; böylece belirli alt tercihler toplam memnuniyet açısından ilk tercihle eşdeğer olabilmektedir. Bu tasarım sayesinde, öğrencinin sosyal bağları ile bireysel tercihleri aynı değerlendirme çerçevesi içerisinde karşılaştırılabilmektedir.

Bu nedenle genişletme fonksiyonu, arkadaşlık toleransı aralığında kalan ve kontenjanı bulunan her tercih için ayrı ayrı state'ler üretir. Örneğin toleransı 2 olan bir öğrenci için ilk üç tercihin her biri uygun ise, mevcut state'ten üç farklı yeni state türetilir. Bu yaklaşım, Greedy algoritmada mümkün olmayan alternatif sosyal yerleşim senaryolarının paralel olarak değerlendirilmesine olanak tanır.

Eğer arkadaşlık toleransı aralığında kontenjanı bulunan hiçbir firma yoksa, algoritma bu noktadan sonra Greedy yaklaşıma benzer bir davranış sergiler. Öğrencinin tolerans dışındaki tercihleri sırayla kontrol edilir ve kontenjanı bulunan ilk firma için tek bir state üretilir. Bu tasarım, öğrencinin sosyal olarak kabul edilebilir aralığın dışına bilinçli olarak itilmesini engellerken, sistemin ilerlemesini de garanti altına alır.

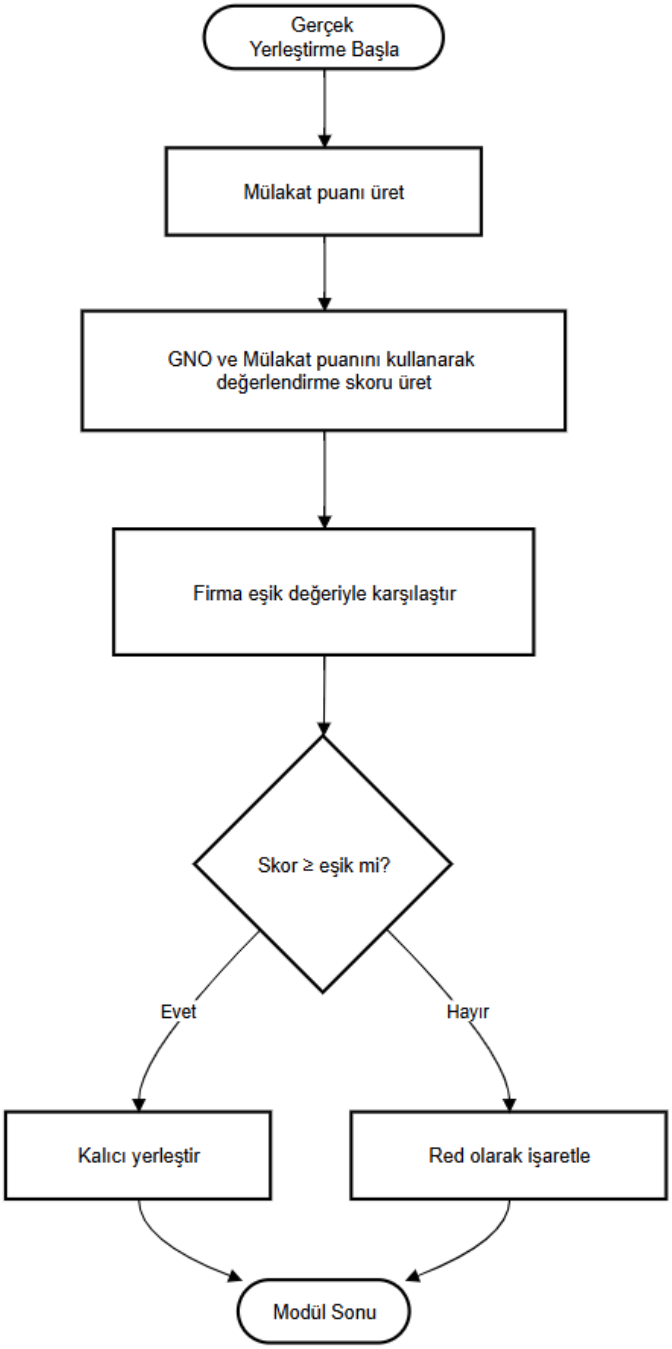
Bu yapı sayesinde heuristic yaklaşım, sosyal faktörleri agresif ve kontrolsüz biçimde optimize etmek yerine, öğrencinin bireysel sınırlarını gözeten dengeli bir arama stratejisi uygular. Bu sayede sistem genelinde daha yüksek toplam memnuniyet üreten çözümlerin ortaya çıkmasını sağlamaktadır.



Şekil 7. State Dallanma Algoritması

D. Gerçek Yerleştirme Modülü - Öğrenci Red Mekanizması

Heuristic yaklaşımda Beam Search modülü tarafından üretilen state'ler, doğrudan kalıcı yerleştirme anlamına gelmemektedir. Her iterasyon sonunda, en yüksek puana sahip kazanan state'e karşılık gelen yerleştirme önerileri, gerçek yerleştirme modülü aracılığıyla firma değerlendirme sürecinden geçirilir. Bu modül, Greedy algoritmada kullanılan firma değerlendirme ve red mekanizması ile aynı yapıyı paylaşmakta olup, algoritmalar arası adil karşılaştırmayı korumak amacıyla ortak bir kontrol noktası olarak tasarlanmıştır.



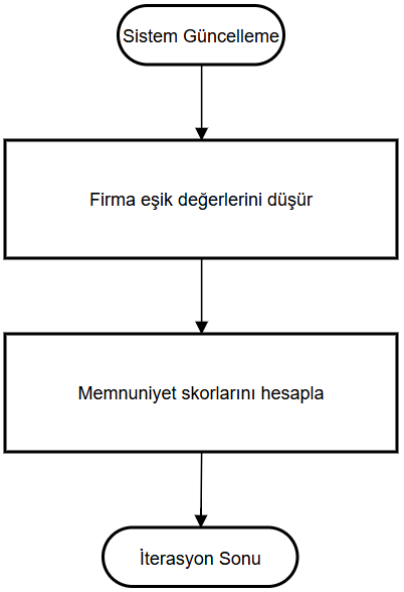
Şekil 8. Gerçek Yerleştirme Algoritması

E. Sistem Güncelleme - Skor Hesaplama

Her oluşturulan state için sezgisel bir skor hesaplanmaktadır. Bu skor, sistemde tanımlanan memnuniyet fonksiyonuna dayanmaktadır ve tercih sırası, iterasyon bilgisi ve arkadaş katkısını birlikte içermektedir.

Heuristic yaklaşımda arkadaş faktörü, yalnızca sonuç değerlendirmesinde değil, doğrudan state'lerin karşılaştırılmasında kullanılmaktadır. Bu sayede, daha yüksek toplam memnuniyet üretecek yerleştirme senaryoları önceliklendirilebilmektedir.

Skor hesaplama işlemi tamamlandıktan sonra, her state'in güncel skoru beam yapısı içinde saklanır.

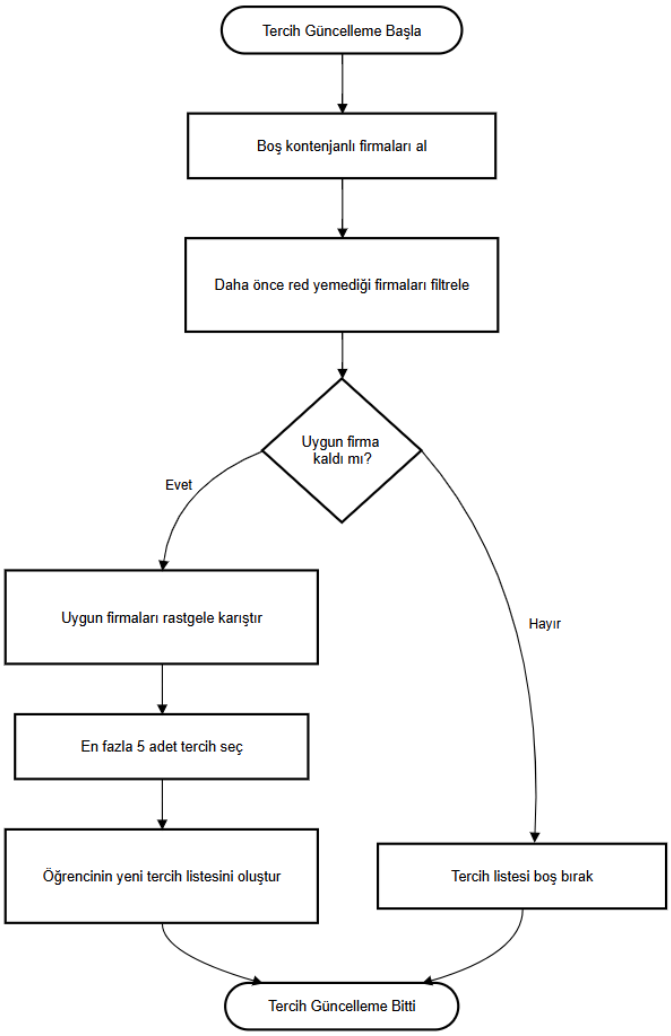


Şekil 9. Sistem Güncelleme Algoritması

F. Tercih Güncelleme Stratejisi

Heuristic yaklaşımda tercih güncelleme işlemi, her iterasyon başında merkezi bir modül aracılığıyla gerçekleştirilir. Öncelikle sistemde boş kontenjana sahip firmalar belirlenir. Ardından öğrencinin daha önce red yediği firmalar bu listeden filtrelenir.

Filtreleme sonrasında uygun firma kalması durumunda, firmalar rastgele karıştırılarak çeşitlilik sağlanır ve en fazla beş adet firma seçilerek öğrencinin yeni tercih listesi oluşturulur. Eğer uygun firma bulunamazsa, öğrencinin tercih listesi boş bırakılır ve öğrenci ilgili iterasyonda yerleştirme sürecine dahil edilmez.



Şekil 10. Tercih Güncelleme Algoritması

G. Zaman Karmaşıklığı

Heuristic yerleştirme yaklaşımı, Beam Search tabanlı bir yapı kullandığından, Greedy algoritmaya kıyasla daha yüksek bir hesaplama maliyetine sahiptir. Algoritmanın zaman karmaşıklığı; beam genişliği (B), iterasyon sayısı (I) ve bir öğrencinin maksimum tercih sayısı (P) gibi parametrelere bağlıdır.

Her iterasyonda en fazla B adet state tutulmakta ve her state için öğrencinin tercih listesi incelenerek yeni state'ler üretilmektedir. Bu nedenle tek bir iterasyonun zaman karmaşıklığı yaklaşık olarak $O(B \cdot P)$ şeklindedir. Algoritma, yerleşmemiş öğrenci kalmayana kadar çalıştığından toplam zaman karmaşıklığı $O(I \cdot B \cdot P)$ olarak ifade edilebilir.

Tercih sayısının sabit ve beam genişliğinin sınırlı tutulması sayesinde, algoritma pratikte kabul edilebilir sürelerde çalışmakta ve Greedy yaklaşımın optimize edemediği küresel memnuniyet kriterlerini iyileştirebilmektedir.

V. Deneysel Çalışmalar

Bu bölümde Greedy ve Beam Search tabanlı heuristic yerleştirme algoritmalarının performansı, kontrollü ve tekrarlanabilir deneyler aracılığıyla karşılaştırmalı olarak incelenmiştir. Deneysel çalışmalar, algoritmaların yalnızca ürettikleri sonuçların kalitesi açısından değil; aynı zamanda problem boyutu büyüdükçe nasıl davrandıkları, sosyal yapıdan ne ölçüde faydalanabildikleri ve parametre değişimlerine karşı duyarlılıkları açısından da değerlendirilmesini amaçlamaktadır. Tüm deneyler, algoritmalar arasında adil bir karşılaştırma yapılabilmesi için aynı veri üretim süreci ve aynı değerlendirme metrikleri kullanılarak gerçekleştirilmiştir

Tüm bu deneysel çalışma adımları algoritmaların genel çalışma mantığına müdahil olmayıp tamamiyle debug işlemlerinin kolay yürütülebilmesi ve görselleştirilmesi amacını taşımaktadır.

A. Veri Setinin Hazırlanması

Deneylerde kullanılan veri setleri, sistemin gerçekçi staj yerleştirme senaryoları üzerinde test edilebilmesi amacıyla otomatik veri üretim modülü kullanılarak oluşturulmuştur. Her deney senaryosunda öğrenci sayısı, firma sayısı ve kontenjan yapısı tutarlı olacak biçimde belirlenmiş; toplam kontenjan her zaman toplam öğrenci sayısına eşit tutulmuştur. Böylece tüm deneyler, sistemin tüm öğrencileri yerleştirmek zorunda olduğu tam dolu senaryolar üzerinde gerçekleştirilmiştir.

Öğrenciler için GNO değerleri, tercih listeleri, arkadaş grupları ve arkadaş toleransı parametreleri rastgele olarak üretilmiştir. Tercih listeleri, sistem tasarımına uygun biçimde en fazla beş firmadan oluşacak şekilde sınırlandırılmıştır. Sosyal yapı, öğrencilerin belirli oranlarda arkadaş gruplarına dahil edilmesiyle modellenmiş ve bu oran deney senaryolarına bağlı olarak kontrollü biçimde değiştirilmiştir. Aynı veri üretim süreci, Greedy ve heuristic algoritmalar için ayrı ayrı ancak özdeş başlangıç koşulları altında uygulanmıştır.

Rastgeleliğin deney sonuçları üzerindeki etkisini azaltmak amacıyla her deney senaryosu birden fazla kez tekrarlanmıştır. Her parametre konfigürasyonu için deneyler 50 bağımsız tekrar ile gerçekleştirilmiş, raporlanan tüm ölçümler bu tekrarların ortalama değerleri üzerinden sunulmuştur. Bu yaklaşım, tekil rastgele dağılımların sonuçları baskın hale getirmesini engellemiş ve elde edilen grafiklerin daha güvenilir olmasını sağlamıştır.

B. Performans Metrikleri

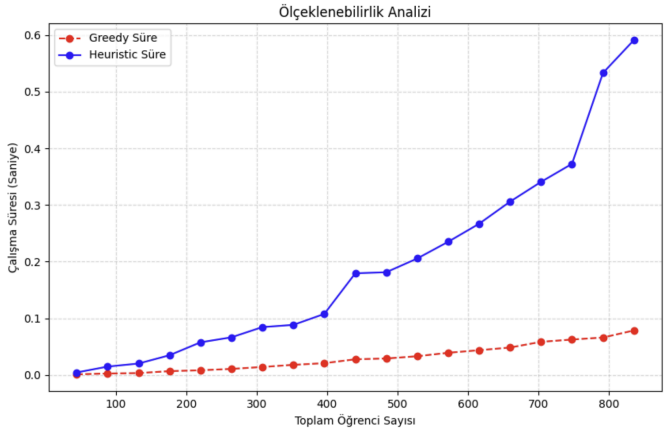
Algoritmaların performansı üç temel metrik üzerinden değerlendirilmiştir. Çalışma süresi, algoritmanın tüm öğrenciler kalıcı olarak yerleşene kadar geçen toplam yürütme süresini ifade etmektedir. Ortalama memnuniyet puanı, kalıcı olarak yerleşmiş öğrencilerin bireysel memnuniyet skorlarının ortalaması olarak hesaplanmıştır. Toplam memnuniyet puanı ise sistem genelinde elde edilen toplam memnuniyet değerini temsil etmekte ve özellikle Beam Width parametresinin çözüm kalitesi üzerindeki etkisini incelemek amacıyla kullanılmıştır. Bu metrikler, algoritmaların hem çözüm kalitesini hem de hesaplama maliyetini birlikte değerlendirmeye olanak tanımaktadır.

C. Deney Senaryoları

Deneyler üç ana senaryo altında gerçekleştirilmiştir. Her senaryoda yalnızca ilgili parametre değiştirilmiş, diğer tüm parametreler sabit tutulmuştur.

1) Öğrenci sayısı – süre

Bu senaryoda algoritmaların ölçeklenebilirliği incelenmiştir. Toplam öğrenci sayısı kademeli olarak artırılmış; kontenjan dağılımı, grup oranı ve Beam Width gibi diğer tüm parametreler sabit tutulmuştur. Her öğrenci sayısı için Greedy ve Heuristic algoritmalar ayrı ayrı 50 kez çalıştırılmış, çalışma süreleri ölçülmüş ve tekrar ortalamaları alınmıştır. Bu analiz, problem boyutu büyüdükçe algoritmaların hesaplama maliyetlerinin nasıl değiştiğini ortaya koymaktadır.



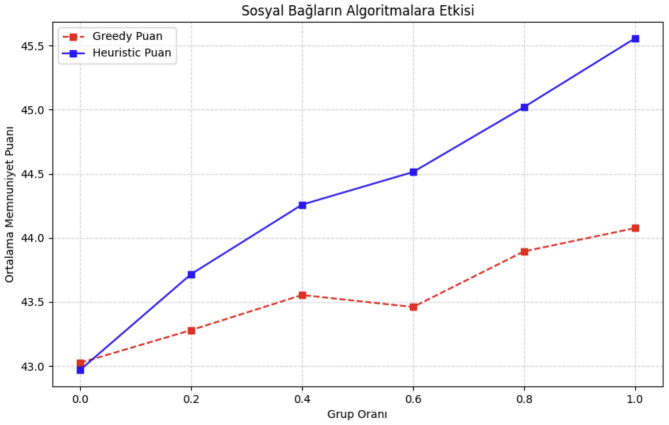
Şekil 11. Öğrenci Sayısı - Süre Karşılaştırması

Şekil 11. incelendiğinde, Greedy algoritmanın çalışma süresinin öğrenci sayısı arttıkça düzenli ve kontrollü bir biçimde yükseldiği görülmektedir. Bu davranış, algoritmanın her iterasyonda sabit uzunlukta tercih listeleri üzerinde çalışması ve tek bir yerleştirme senaryosunu takip etmesiyle doğrudan ilişkilidir. Dolayısıyla Greedy yaklaşım, problem boyutu büyüse dahi öngörülebilir bir hesaplama maliyeti sunmakta ve büyük ölçekli senaryolarda pratik bir çözüm olarak öne çıkmaktadır.

Heuristic yaklaşımda ise öğrenci sayısındaki artışa paralel olarak çalışma süresinin daha hızlı yükseldiği dikkat çekmektedir. Bunun temel nedeni, Beam Search yapısı içerisinde aynı anda birden fazla olası yerleştirme senaryosunun değerlendirilmesi ve her state için ek skor hesaplamalarının yapılmasıdır. Bu sonuçlar, heuristic yöntemin daha yüksek çözüm kalitesi sağlarken zaman maliyeti açısından Greedy'ye göre daha pahalı olduğunu göstermektedir.

2) Grup oranı – ortalama memnuniyet puanı

Bu senaryoda sosyal bağların algoritmaların ürettiği memnuniyet üzerindeki etkisi değerlendirilmiştir. Firma sayısı sabit tutulurken, grup oranı parametresi kademeli olarak artırılmıştır. Grup oranı, öğrencilerin ne kadarının sosyal gruplara dahil olduğunu temsil etmektedir. Her grup oranı değeri için algoritmalar tekrar çalıştırılmış ve ortalama memnuniyet puanları hesaplanmıştır. Bu deney, özellikle arkadaş faktörünü aktif biçimde karar sürecine dahil eden heuristic yaklaşımın sosyal yoğunluk arttıkça nasıl davrandığını analiz etmeyi amaçlamaktadır.



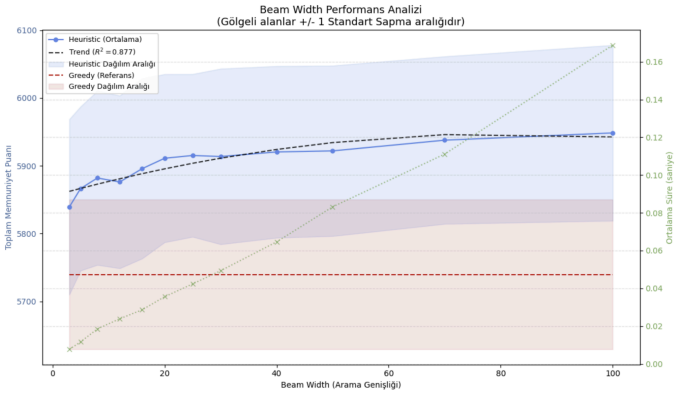
Şekil 12. Grup Oranı - Memnuniye Puanı Karşılaştırması

Şekil 12. incelendiğinde, grup oranı arttıkça Greedy algoritmanın ürettiği ortalama memnuniyet puanının sınırlı bir artış gösterdiği görülmektedir. Bunun nedeni, Greedy yaklaşımda arkadaşlık bilgisinin yerleştirme kararlarını etkilememesi ve yalnızca sonuç aşamasında puanlamaya yansımasıdır. Sosyal bağlar yoğunlaştıkça, Greedy algoritma bu bilgiyi yerleştirme sürecine yönlendiremediği için potansiyel memnuniyet artışlarını kaçırmaktadır.

Heuristic yaklaşımda ise grup oranındaki artışa paralel olarak ortalama memnuniyet puanında belirgin ve tutarlı bir yükseliş gözlemlenmektedir. Beam Search mekanizması, arkadaş toleransı ve sosyal bağları aktif olarak dallanma ve state değerlendirme sürecine dahil ettiği için, sosyal yoğunluk arttıkça daha avantajlı yerleşim senaryoları üretebilmektedir. Bu grafik, sosyal faktörlerin güçlü olduğu ortamlarda heuristic yaklaşımın Greedy algoritmaya kıyasla neden daha yüksek çözüm kalitesi sunduğunu açık biçimde ortaya koymaktadır.

3) Beam width – toplam memnuniyet puanı

Bu senaryoda farklı beam width değerleri denenmiş ve bu parametrenin toplam memnuniyet skoru üzerindeki etkisi deneysel olarak analiz edilmiştir. Beam width parametresi, çözüm kalitesi ile hesaplama maliyeti arasındaki temel denge unsuru olarak ele alınmıştır.



Şekil 13. Beam Width - Memnuniyet Puanı Karşılaştırması

Her Beam Width değeri için rastgele veriler üretilerek 50 kere toplam memnuniyet skoru ölçülmüş, bu puanların

ortalaması alınmış ve elde edilen sonuçlar karşılaştırılmıştır. Test edilen makul öğrenci sayısı göz önünde bulundurulduğunda beam width'in 40'tan sonraki değerlerinde anlamlı bir artış görülmemiş, optimum değer olarak 20-40 aralığı seçilmiştir.

D. Grafik Analizleri

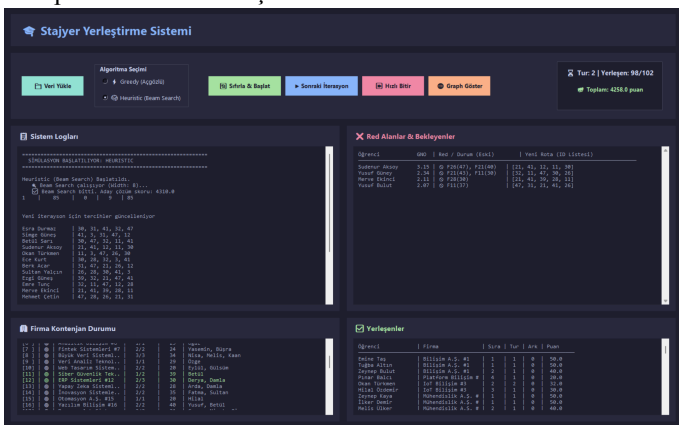
Ölçeklenebilirlik grafikleri, Greedy algoritmanın düşük hesaplama maliyetine sahip olduğunu; heuristic yaklaşımın ise daha yüksek ancak kontrol edilebilir bir çalışma süresiyle daha kaliteli çözümler ürettiğini göstermektedir. Sosyal etki analizleri, arkadaşlık yapılarının yoğunlaştığı senaryolarda heuristic yaklaşımın belirgin bir üstünlük sağladığını ortaya koymaktadır. Beam Width analizleri ise çözüm kalitesi ile hesaplama maliyeti arasındaki ilişkiyi açık biçimde göstermektedir.

VI. GUI ve Görselleştirme

Bu görselleştirme katmanı, algoritmanın mantıksal işleyişine doğrudan etki eden bir bileşen olmayıp; geliştirme, hata ayıklama (debug) ve sonuçların yorumlanmasını kolaylaştırmak amacıyla tasarlanmıştır.

A. Tkinter Arayüzü

Geliştirilen simülasyon yazılımı, kullanıcı etkileşimini en üst düzeye çıkarmak ve karmaşık yerleştirme süreçlerini anlaşılır kılmak amacıyla Python'un standart GUI kütüphanesi olan Tkinter kullanılarak tasarlanmıştır. Arayüz tasarımında ModernTheme sınıfı üzerinden "karanlık mod" (dark mode) renk paleti tercih edilmiştir.



Şekil 14. Tkinter GUI Arayüzü

Arayüz, işlevsellik bakımından üç ana bloğa ayrılmıştır:

1) Kontrol Paneli:

Ekranın üst kısmında yer alan bu panel, kullanıcının simülasyonu yönetmesini sağlar. Kullanıcı, "Algoritma Seçimi" bölümünden Greedy veya Heuristic (Beam Search) yöntemlerinden birini seçebilir. "Adım Adım İlerlet" (Sonraki İterasyon) butonu ile algoritmanın her bir turu manuel olarak incelenebilirken, "Hızlı Bitir" butonu ile simülasyon otomatik olarak tamamlanabilmektedir. Ayrıca anlık durum paneli, o anki tur sayısını, toplam yerleşen öğrenci sayısını ve sistemin toplam memnuniyet puanını dinamik olarak göstermektedir.

2) Log ve Firma Durum Panelleri:

Sol blokta yer alan "Sistem Logları" paneli, algoritmanın arka planda aldığı kararları ("Beam Search çalışıyor", "Tercihler güncelleniyor" vb.) metin tabanlı olarak raporlar. Heuristic yerleştirme sırasında o iterasyonda yerleşemeyen öğrencilerin sıradaki iterasyon için yaptıkları güncel tercihler de bu alanda gösterilir. Hemen altında bulunan "Firma Kontenjan Durumu" paneli ise firmaların doluluk oranlarını, eşik değerlerini ve o an firmada bulunan öğrencileri listeler. Müsait firmalar yeşil, dolu firmalar ise görsel ayırt ediciliği sağlamak adına gri renkte kodlanmıştır.

3) Öğrenci Durum Panelleri:

Sağ blokta "Red Alanlar & Bekleyenler" paneli bulunmaktadır. Bu panel, reddedilen öğrencilerle beraber "Red Tercih Listesi" bilgisini de sunar. Böylece kullanıcının, öğrencilerin hangi tercihleri ile reddedildiğini görmesini sağlar. "Yerleşenler" paneli ise kalıcı ataması yapılan öğrencilerin hangi iterasyonda kaçıncı tercihlerine yerleştiklerini, getirdikleri memnuniyet puanlarını ve aynı firmaya yerleşen arkadaş sayılarını listeler.

B. Beam Search Ağacının Görselleştirilmesi

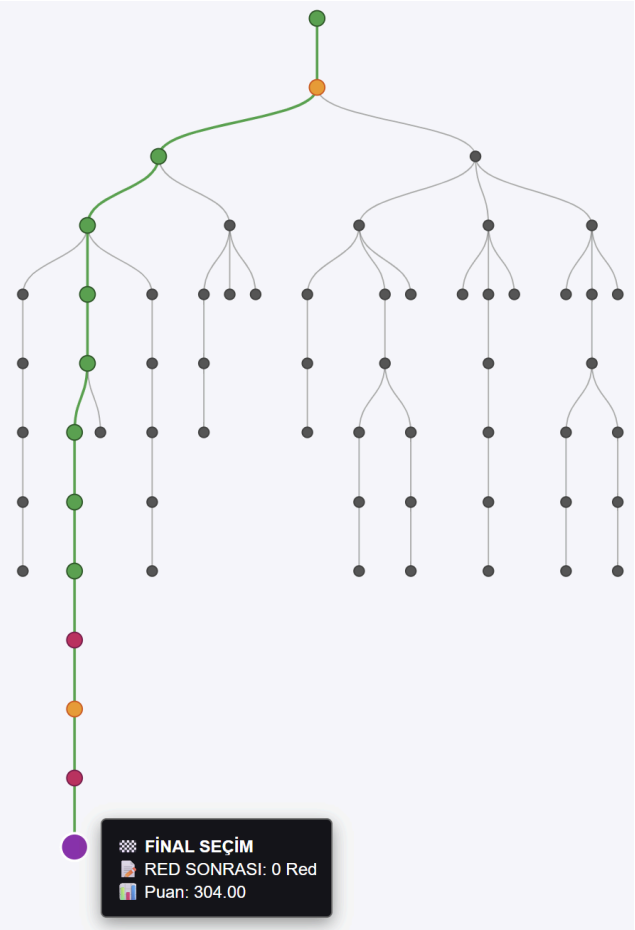
Heuristic yaklaşımda kullanılan Beam Search algoritması, her iterasyonda birden fazla olasılığı (state) değerlendirip dallanarak ilerlediği için, sürecin sadece metin tabanlı loglarla takip edilmesi zorlaşmaktadır. Bu problemi çözmek ve algoritmanın karar uzayını görselleştirmek amacıyla harici bir görselleştirme modülü geliştirilmiştir.

Kod içerisinde yer alan *generate_html_graph()* fonksiyonu, simülasyon tamamlandığında bellekteki tüm state geçmişini (state history) analiz eder ve ebeveyn-çocuk ilişkilerini (parent-child nodes) kurar. Elde edilen bu ağaç yapısı, web tabanlı veri görselleştirme kütüphanesi olan D3.js formatına dönüştürülerek .html uzantılı bir dosya olarak dışa aktarılır.

Oluşturulan bu görselleştirmede:

- Düğümler (Nodes): Her bir düğüm, o andaki kısmi yerleştirme durumunu (bir öğrencinin bir firmaya atanması veya pas geçilmesi) temsil eder.
- Renk Kodlaması: Algoritmanın en yüksek skoru elde ederek seçtiği "kazanan yol" (winning path) yeşil renk ile vurgulanırken, daha düşük skor ürettiği için budanan (pruned) veya seçilmeyen dallar gri renkle gösterilir. Final düğümü ise mor ile işaretlenerek sonucun ayırt edilmesi sağlanır.
- Etkileşim: Kullanıcı, fare ile düğümlerin üzerine geldiğinde (tooltip), o state'in ürettiği anlık skoru ve log detayını görebilmektedir.

Bu görselleştirme aracı, özellikle Beam Width parametresinin artırıldığı durumlarda algoritmanın çözüm uzayını nasıl taradığını analiz etmek için kritik bir hata ayıklama ve gözlem imkanı sunmaktadır. Ağaç yapısı ve kullanılan kütüphaneler heuristic algoritmaya doğrudan bir katkı sağlamamakta, sadece ağaç yapısının görsellerle daha iyi anlaşılabilmesi ve debug işlemlerinin kolayca yapılabilmesi için tasarlanmıştır.



Şekil 15. Örnek Beam Search Çıktısı (Beam width=8)

VII. Sonuç ve Tartışma

Bu çalışmada, stajyer yerleştirme problemi için iki farklı algoritmik yaklaşım geliştirilmiş ve karşılaştırmalı olarak incelenmiştir. Deterministik bir yöntem olan Greedy algoritma ile sezgisel ve arama tabanlı Beam Search yaklaşımı; yerleştirme kalitesi, hesaplama süresi ve sosyal faktörlerin etkisi açısından değerlendirilmiştir. Her iki yöntemin de %100 yerleşme oranına ulaşması sağlanmış; ancak çözüm kalitesi ve elde edilen toplam memnuniyet skoru açısından farklılıklar gözlemlenmiştir.

A. Algoritma Performansları

Greedy algoritma, düşük hesaplama maliyeti ve öngörülebilir davranışı sayesinde özellikle büyük öğrenci kümelerinde hızlı ve kararlı sonuçlar üretmiştir. Öğrencilerin GNO'ya göre sıralanarak yerleştirilmesi, sistemin erken iterasyonlarda büyük ölçüde dengelenmesini sağlamış ve çalışma süresinin doğrusal biçimde artmasına neden olmuştur. Bununla birlikte Greedy yaklaşımın deterministik yapısı, yerleştirme sürecinde yalnızca yerel kararlar almasına yol açmış ve küresel memnuniyet açısından daha iyi olabilecek alternatif senaryoların değerlendirilmesini engellemiştir.

Heuristic yaklaşım ise Beam Search yapısı sayesinde birden fazla olası yerleşim senaryosunu eş zamanlı olarak değerlendirebilmiştir. Bu durum, özellikle arkadaşlık ilişkilerinin yoğun olduğu veri setlerinde toplam memnuniyet skorunun anlamlı biçimde artmasını sağlamıştır. Ancak bu kazanım, artan hesaplama süresi ile birlikte gelmiştir. Beam width parametresi büyüdükçe çözüm kalitesi artmış; buna karşılık algoritmanın çalışma süresi de belirgin şekilde yükselmiştir. Bu sonuç, sezgisel arama yöntemlerinde beklenen kalite–maliyet dengesini açık biçimde ortaya koymaktadır.

B. Karşılaştırmalı Değerlendirme

Deneysel sonuçlar, Greedy ve Heuristic yaklaşımların birbirine alternatif değil, farklı önceliklere hizmet eden yöntemler olduğunu göstermektedir. Greedy algoritma, sosyal etkileşimlerin sınırlı olduğu veya yalnızca akademik başarı odaklı sistemlerde yeterli bir çözüm sunmaktadır. Buna karşılık sosyal bağların güçlü olduğu senaryolarda, Greedy yaklaşımın göz ardı ettiği arkadaş faktörü, toplam memnuniyet üzerinde belirleyici bir rol oynamaktadır.

Beam Search tabanlı heuristic yaklaşımda arkadaşlık bilgisi, yalnızca sonuçların değerlendirilmesinde değil, doğrudan yerleştirme kararlarının şekillenmesinde aktif olarak kullanılmıştır. Bu sayede, bireysel olarak daha düşük öncelikli görünen bazı yerleştirmelerin, sistem genelinde daha yüksek memnuniyet ürettiği gözlemlenmiştir. Özellikle grup oranının arttığı deney senaryolarında, heuristic yaklaşımın Greedy'ye kıyasla daha tutarlı ve dengeli çözümler ürettiği ortaya konmuştur.

Öte yandan, heuristic yöntemin her koşulda üstün olduğu söylenemez. Beam width parametresinin yetersiz seçilmesi durumunda, algoritma Greedy'ye yakın davranışlar sergileyebilmekte; aşırı büyük değerlerde ise hesaplama maliyeti pratik kullanım sınırlarını zorlayabilmektedir. Bu durum, heuristic yaklaşımın dikkatli parametre ayarı gerektirdiğini göstermektedir.

C. Gelecek Çalışma Önerileri

Bu çalışma, merkezi yerleştirme problemlerinde sosyal faktörlerin dikkate alınmasının çözüm kalitesini anlamlı biçimde artırabileceğini göstermektedir. Çalışmanın uyarlanabileceği bir başka alan da üniversitelerdeki seçmeli ders seçim sistemleridir. Öğrencilerin yalnızca ders tercihleri değil, birlikte ders almak istedikleri arkadaş grupları da değerlendirmeye dahil edilerek daha dengeli sınıf dağılımları elde edilebilir.

Benzer biçimde, işe alım ve stajyer seçme süreçlerinde adayların sosyal ağ yapıları dikkate alınarak ekip uyumu odaklı yerleştirme sistemleri geliştirilebilir. Özellikle proje bazlı çalışmalarda, bireysel başarıdan ziyade grup içi etkileşimin ön plana çıktığı durumlar için bu tür sezgisel yaklaşımlar değerli bir alternatif sunmaktadır.

Ayrıca, Beam Search yapısının farklı sezgisel skor fonksiyonları ile zenginleştirilmesi, adaptif beam width mekanizmalarının kullanılması ve paralel hesaplama teknikleri ile performansın artırılması, bu çalışmanın doğal devamı niteliğinde araştırma konuları olarak değerlendirilebilir.

KAYNAKLAR

[1] GeeksforGeeks, "Introduction to Beam Search Algorithm," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/introduction-to-beam-search-algorithm/. Accessed: Jan. 3, 2026